

Author: Collin Aslin Dsouza

In [1]: *#Importing the Libraries*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.ensemble import GradientBoostingClassifier
```

In [15]: *# Load the dataset*

```
df = pd.read_csv("advertising.csv")
```

In [16]: *#Explore the first few rows of the dataset*

```
df.head()
```

Out[16]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Male	Country	Timestamp	Clicked on Ad
0	68.95	35	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	0	Tunisia	2016-03-27 00:53:11	0
1	80.23	31	68441.85	193.77	Monitored national standardization	West Jodi	1	Nauru	2016-04-04 01:39:02	0
2	69.47	26	59785.94	236.50	Organic bottom-line service-desk	Davidton	0	San Marino	2016-03-13 20:35:42	0
3	74.15	29	54806.18	245.89	Triple-buffered reciprocal time-frame	West Terrifurt	1	Italy	2016-01-10 02:31:19	0
4	68.37	35	73889.99	225.58	Robust logistical utilization	South Manuel	0	Iceland	2016-06-03 03:36:18	0

In [17]: *#Explore the data types and check for missing values*

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Daily Time Spent on Site              1000 non-null   float64
1   Age                                    1000 non-null   int64
2   Area Income                           1000 non-null   float64
3   Daily Internet Usage                  1000 non-null   float64
4   Ad Topic Line                         1000 non-null   object
5   City                                   1000 non-null   object
6   Male                                   1000 non-null   int64
7   Country                               1000 non-null   object
8   Timestamp                             1000 non-null   object
9   Clicked on Ad                         1000 non-null   int64
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB
```

The dataset exhibits excellent data quality with no missing values across all 1000 entries. The data types are suitable for a classification task, with a balance of numerical features (like Area Income and Age) and categorical/text features (like Country and Ad Topic Line), and a clear integer target variable (Clicked on Ad).

In [18]: *#Summary Statistics*

```
df.describe()
```

Out[18]:	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Male	Clicked on Ad
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	65.000200	36.009000	55000.000080	180.000100	0.481000	0.500000
std	15.853615	8.785562	13414.634022	43.902339	0.499889	0.500250
min	32.600000	19.000000	13996.500000	104.780000	0.000000	0.000000
25%	51.360000	29.000000	47031.802500	138.830000	0.000000	0.000000
50%	68.215000	35.000000	57012.300000	183.130000	0.000000	0.500000
75%	78.547500	42.000000	65470.635000	218.792500	1.000000	1.000000
max	91.430000	61.000000	79484.800000	269.960000	1.000000	1.000000

On average, users spend about 65 minutes daily on the site and 180 minutes using the internet, with a mean age of 36 and an average area income of \$55,000. The binary columns show that the population is almost equally split between Male 48.1% and Female, and that 50% of users clicked on the ad.

```
In [19]: # 1. Define Features (X) and Target (y)
# Using only numerical features for simplicity
feature_cols = [
    'Daily Time Spent on Site',
    'Age',
    'Area Income',
    'Daily Internet Usage',
    'Male'
]
target_col = 'Clicked on Ad'

X = df[feature_cols]
y = df[target_col]
```

```
In [20]: numerical_features = [
    'Age',
    'Daily Time Spent on Site',
    'Area Income'
]

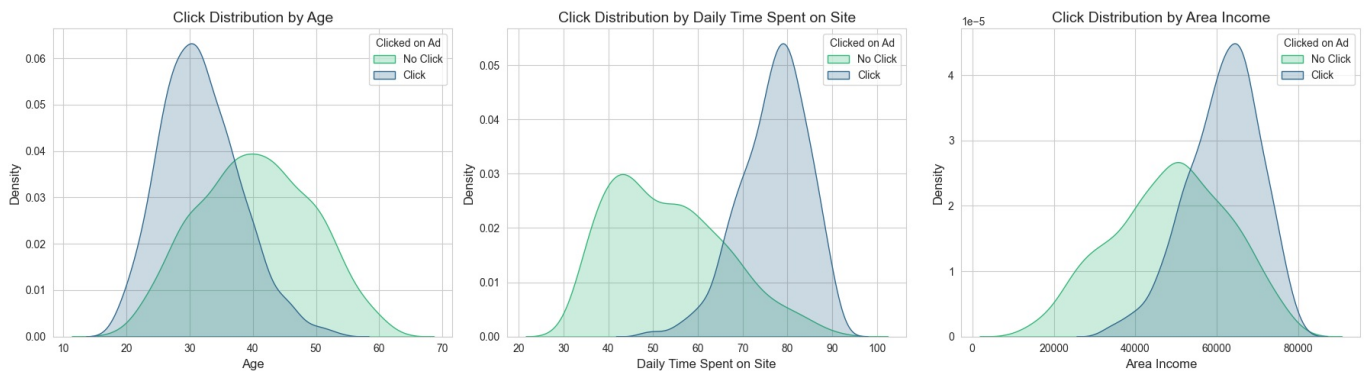
# --- Generate Visualizations ---
import seaborn as sns
# Set up the figure size and style
sns.set_style('whitegrid')
plt.figure(figsize=(18, 5)) # Use a wide figure for 3 plots

for i, feature in enumerate(numerical_features):
    # Create a subplot for each feature
    plt.subplot(1, 3, i + 1)

    # Generate a KDE plot to compare the distribution of the feature
    # for customers who clicked (1) vs. those who didn't (0).
    sns.kdeplot(
        data=df,
        x=feature,
        hue='Clicked on Ad',
        fill=True,
        common_norm=False,
        palette='viridis' # Use a color palette to distinguish the classes
    )

    # Add titles and labels for clarity
    plt.title(f'Click Distribution by {feature}', fontsize=14)
    plt.xlabel(feature, fontsize=12)
    plt.ylabel('Density', fontsize=12)
    plt.legend(title='Clicked on Ad', labels=['No Click', 'Click'])

# Adjust layout to prevent plot overlap
plt.tight_layout()
plt.show()
```



1. Click Distribution by Age

Users who clicked on the ad (Blue) tend to be younger, with their peak density centered around the late 20s to early 30s. Users who did not click (Green) are generally older, peaking in the late 30s to early 40s.

2. Click Distribution by Daily Time Spent on Site

Users who clicked (Blue) spend a higher amount of time on the site, with the distribution heavily weighted towards 70-95 minutes. Users who did not click (Green) spend less time on the site, primarily concentrated between 35-65 minutes.

3. Click Distribution by Area Income

Users who clicked (Blue) tend to have higher Area Income, with their distribution peaking around 60,000-75,000. Users who did not click (Green) are concentrated in the lower-middle income range, peaking around 45,000-60,000.

```
In [21]: # 2. Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [22]: # 3. Initialize and train the Gradient Boosting Classifier
gb_model = GradientBoostingClassifier(
    n_estimators=100,
    max_depth=3,
    random_state=42
)
gb_model.fit(X_train, y_train)

# 4. Make predictions and Evaluate the model
y_pred = gb_model.predict(X_test)
report = classification_report(y_test, y_pred)

print(report)
```

	precision	recall	f1-score	support
0	0.91	0.93	0.92	89
1	0.94	0.93	0.94	111
accuracy			0.93	200
macro avg	0.93	0.93	0.93	200
weighted avg	0.93	0.93	0.93	200

Gradient Boosting

The model exhibits excellent overall performance with an accuracy of 93% on the test set. Crucially, the model shows high and balanced predictive power for both classes, as indicated by the consistently high Precision and Recall scores (≥ 0.91) across both Class 0 and Class 1.

```
In [23]: ##KNN MODEL
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report

# Load the dataset
df1 = pd.read_csv("advertising.csv")

# 1. Define Features (X) and Target (y)
feature_cols = [
    'Daily Time Spent on Site',
    'Age',
```

```

    'Area Income',
    'Daily Internet Usage',
    'Male'
]
target_col = 'Clicked on Ad'

X1 = df1[feature_cols]
y1 = df1[target_col]

# 2. Split the data into training and testing sets (80% train, 20% test)
X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)

# 3. Standardize the data
scaler = StandardScaler()
X1_train_scaled = scaler.fit_transform(X1_train)
X1_test_scaled = scaler.transform(X1_test)

# 4. Initialize and train the K-Nearest Neighbors (KNN) Classifier
# n_neighbors=5 is a common choice for k
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X1_train_scaled, y1_train)

# 5. Make predictions and Evaluate the model
y1_pred = knn_model.predict(X1_test_scaled)
report = classification_report(y1_test, y1_pred)

print(report)

```

	precision	recall	f1-score	support
0	0.89	0.96	0.92	89
1	0.96	0.90	0.93	111
accuracy			0.93	200
macro avg	0.92	0.93	0.92	200
weighted avg	0.93	0.93	0.93	200

The KNN model

The KNN Model achieves a strong overall accuracy of 93 on the test set. It exhibits a slight preference for classifying the "No Click" group (Class 0) correctly, evidenced by its higher recall (0.96), while maintaining a high precision (0.96) for the "Click" group (Class 1).

```

In [24]: ## Random forest and decision tree model
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import pandas as pd

# Load the dataset (assuming 'df' is not already defined)
df2 = pd.read_csv("advertising.csv")

# 1. Define Features (X) and Target (y)
feature_cols2 = [
    'Daily Time Spent on Site',
    'Age',
    'Area Income',
    'Daily Internet Usage',
    'Male'
]
target_col2 = 'Clicked on Ad'

X2 = df[feature_cols2]
y2 = df[target_col2]

# 2. Split the data into training and testing sets
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
)

# --- DECISION TREE CLASSIFIER ---
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X2_train, y2_train)
dt_y_pred = dt_model.predict(X2_test)
dt_report = classification_report(y2_test, dt_y_pred)

print("--- Decision Tree Classification Report ---")
print(dt_report)

```

```
# --- RANDOM FOREST CLASSIFIER ---
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y2_train)
rf_y_pred = rf_model.predict(X2_test)
rf_report = classification_report(y2_test, rf_y_pred)

print("--- Random Forest Classification Report ---")
print(rf_report)
```

```
--- Decision Tree Classification Report ---
              precision    recall  f1-score   support

         0       0.91      0.89      0.90         89
         1       0.91      0.93      0.92        111

   accuracy          0.91
  macro avg          0.91
 weighted avg          0.91

--- Random Forest Classification Report ---
              precision    recall  f1-score   support

         0       0.91      0.93      0.92         89
         1       0.94      0.93      0.94        111

   accuracy          0.93
  macro avg          0.93
 weighted avg          0.93
```

Decision Tree Classification Report

The Decision Tree model achieved a good overall accuracy of 91, showing slightly better recall for Class 1 (Clicked on Ad) than Class 0 (No Click). Its performance is consistent across both classes, with F1-scores of 0.90 and 0.92.

Random Forest Classification Report

The Random Forest model is more accurate, achieving 93, and demonstrates higher and more balanced predictive power than the Decision Tree. It shows excellent performance for both classes, especially for Class 1, which has a higher precision of 0.94.

Evaluation and Choosing the best model

Model |Accuracy |Class 0 (No Click) F1-Score |Class 1 (Click) F1-Score

Gradient Boosting |0.93 |0.92 |0.94

Random Forest |0.93 |0.92 |0.94

KNN |0.93 |0.92 |0.93

Decision Tree |0.91 |0.90 |0.92

Top Performance:

Gradient Boosting, Random Forest, and KNN are the top-performing models, each achieving a high and consistent 93 accuracy.

Ensemble Advantage:

Ensemble methods (Random Forest and Gradient Boosting) show superior prediction of the positive class (Click), achieving the highest F1-score of 0.94 for Class 1.

Feature Predictiveness:

The consistently high accuracy across models confirms that features like age, daily time on site, and area income are highly predictive of ad click behavior.

Recommended Model:

Random Forest is recommended for deployment due to its optimal balance of 93 accuracy, high stability, and faster training speed compared to Gradient Boosting, minimizing both wasted ad spend (false positives) and missed opportunities (false negatives).

Why Random Forest is Optimal for Business Use

Choosing Random Forest over Gradient Boosting is often the optimal business decision due to its excellent balance of high performance and resource efficiency. It allows a business to gain maximum predictive benefit while utilizing minimum computational resources.

Here are the key points on why Random Forest (RF) is resource-efficient and highly practical for deployment:

Parallelization of Training:

Random Forest Advantage: Random Forest is an embarrassingly parallel algorithm. Each individual decision tree is trained independently and simultaneously , which significantly speeds up the training process, especially on modern systems with multiple cores or distributed computing resources.

GB Disadvantage: Gradient Boosting (GB) is sequential; each new tree corrects the errors of the previous one, meaning trees cannot be trained at the same time, making the training process much slower.

Reduced Overfitting and Stability:

Random Forest Advantage: Random Forest builds trees from bootstrapped samples of the data and uses a random subset of features for each split. This inherent randomness makes the model highly robust, less prone to overfitting, and ensures good generalization to new data without extensive tuning.

Business Benefit: A stable model requires less maintenance and fewer retraining cycles, which translates directly into lower operational costs and engineering hours.

Faster Prediction/Inference Speed:

Random Forest Advantage: Once trained, Random Forest models can provide predictions very quickly. The prediction process is simpler than in GB because it just averages the output of many independent trees.

Business Benefit: Faster prediction times are crucial for a dynamic advertising environment (like real-time bidding or page load), ensuring minimal latency when deciding whether to show an ad.

Fewer Hyperparameter Needs:

Random Forest Advantage: Random Forest typically requires tuning only a few core hyperparameters (like the number of trees and the maximum features per split) to achieve high performance.

Business Benefit: This simplicity means less time and computational power are spent on extensive grid search or complex optimization, reducing the time-to-deployment.

In summary, while Gradient Boosting often achieves marginally better peak performance, Random Forest delivers near-identical top-tier accuracy (93%) with lower computational costs, faster deployment, and superior stability, making it the most robust and financially sound choice.

In []:

Processing math: 100%